

넘파이 (NumPy) (1)

2026-1학기

ICT융합학부 컴퓨터공학트랙

김대환

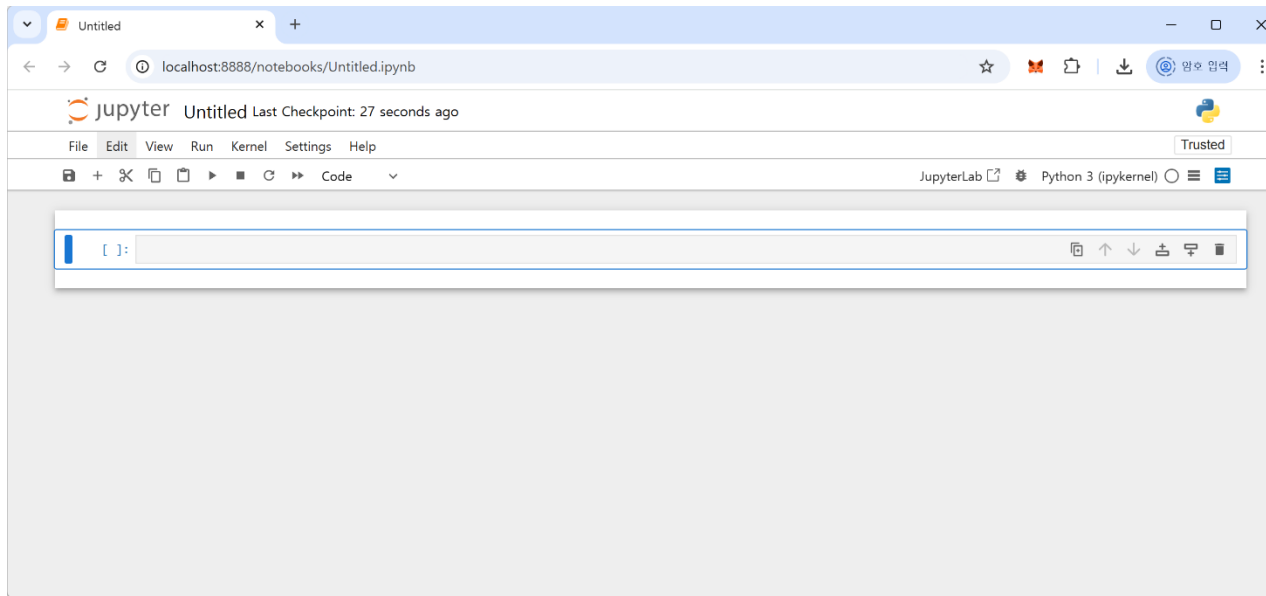
목차

1. 파이썬 사용 환경 (Jupyter Notebook)
2. 넘파이 (NumPy) 소개
3. 파이썬 데이터 타입 이해하기
4. NumPy 배열의 기초
5. NumPy 배열연산: 유니버설 함수
6. 집계: 최솟값, 최댓값 등

파이썬 사용 환경

파이썬 사용 환경 (1)

■ 주피터 노트북 (Jupyter Notebook) 사용



■ 직접 설치

- <https://jupyter.org/install>
- 설치 방법

Jupyter Notebook

Install the classic Jupyter Notebook with:

```
pip install notebook
```

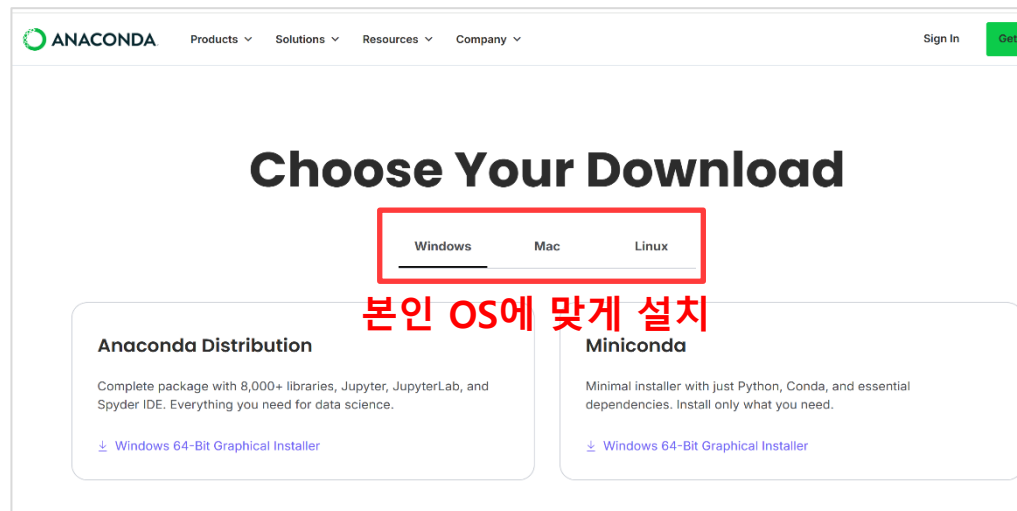
To run the notebook:

```
jupyter notebook
```

파이썬 사용 환경 (2)

■ 아나콘다(Anaconda) 설치

- <https://www.anaconda.com/download>
- 설치 옵션
 - Just Me (Recommended) 선택
 - Install Location: 기본 경로 사용
 - 아래 옵션 체크
 - Add Anaconda to PATH
 - Register Anaconda as default Python



넘파이 (NumPy) 소개

넘파이 (NumPy) 소개 (1)

- 넘파이 (NumPy)

- 넘파이(NumPy): Numerical Python의 약자
- Python에서 숫자 데이터를 빠르고 편하게 다루기 위한 핵심 라이브러리
- **다차원 배열(ndarray)** 을 중심으로 동작하며, 배열에 대한 수학 연산, 통계 계산, 정렬, 형태 변환 등을 효율적으로 처리

- 넘파이 (NumPy) vs 파이썬 리스트 (List)

- 넘파이 배열은 파이썬 리스트와 비슷해 보이지만, 배열의 규모가 커질수록 데이터 저장/처리에 훨씬 효율적
- NumPy 배열은 **같은 자료형의 값들을 연속된 구조로 저장**하고, 많은 연산을 **미리 컴파일된 코드 기반으로 처리**하기 때문에 성능상 유리

넘파이 (NumPy) 소개 (2)

- 넘파이 (NumPy) 핵심 : 배열 (Array)

- n차원 배열 객체(ndarray)
- 1차원, 2차원, 3차원 이상의 데이터를 표현할 수 있는 구조
- (예)
 - 1차원 배열: 시험 점수 목록
 - 2차원 배열: 학생별 과목 점수표
 - 3차원 배열: 여러 장의 이미지 데이터

```
import numpy as np

x = np.array([10, 20, 30, 40])
print(x)
```

```
[10 20 30 40]
```

```
import numpy as np
y = np.array([[1, 2, 3],
              [4, 5, 6]])
print(y)
```

```
[[1 2 3]
 [4 5 6]]
```


파이썬 데이터 타입 이해하기

파이썬 데이터 타입 이해하기 (1)

- 파이썬
 - 동적 타입 언어 (Dynamic type)
 - 변수 선언 시 데이터 타입을 지정하지 않음

```
/* C code */  
int result = 0;  
for(int i=0; i<100; i++){  
    result += i;  
}
```

```
# Python code  
result = 0  
for i in range(100):  
    result += i
```

VS

```
/* C code */  
int x = 4;  
x = "four"; // FAILS
```

```
# Python code  
x = 4  
x = "four"
```

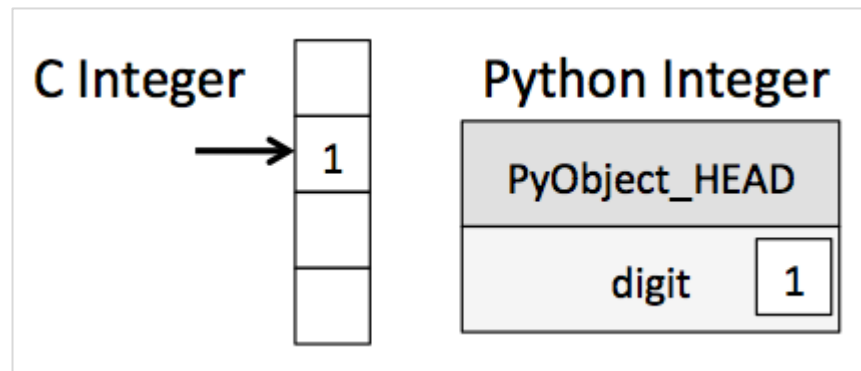
파이썬 데이터 타입 이해하기 (2)

■ 파이썬 정수

- 단순 숫자가 아니라 객체 구조

```
struct _longobject {  
    long ob_refcnt;  
    PyTypeObject *ob_type;  
    size_t ob_size;  
    long ob_digit[1];  
};
```

- ob_refcnt: 메모리 할당과 해제를 처리할 수 있게 돕는 참조 횟수
- ob_type: 변수 타입을 인코딩
- ob_size: 다음 데이터 멤버의 크기를 지정
- ob_digit: 파이썬 변수가 나타내는 실제 정숫값을 포함



C 정수와 파이썬 정수의 차이

파이썬 데이터 타입 이해하기 (3)

■ 파이썬 리스트

■ 다양한 타입 저장 가능

```
In [1]: L = list(range(10))  
L
```

```
Out[1]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
In [2]: type(L[0])
```

```
Out[2]: int
```

```
In [3]: L2 = [str(c) for c in L]  
L2
```

```
Out[3]: ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
```

```
In [4]: type(L2[0])
```

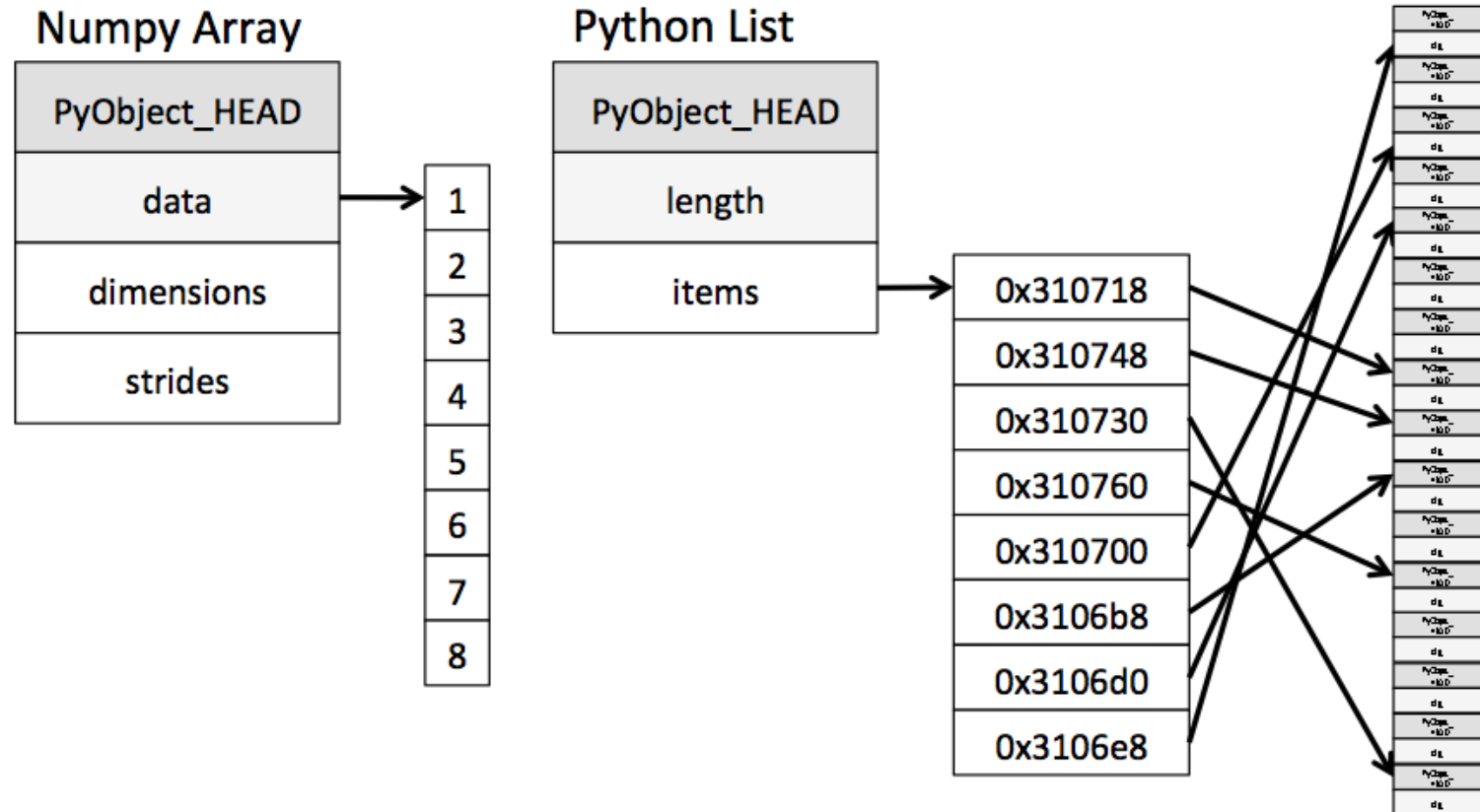
```
Out[4]: str
```

```
In [5]: L3 = [True, "2", 3.0, 4]  
[type(item) for item in L3]
```

```
Out[5]: [bool, str, float, int]
```

파이썬 데이터 타입 이해하기 (4)

- 파이썬 리스트와 넘파이 배열 차이



파이썬 데이터 타입 이해하기 (5)

■ 파이썬 고정 타입 배열 생성

- 고정 타입 데이터 버퍼를 저장하는 방식 제공
- 내장 array 모듈을 사용

```
In [6]: import array
        L = list(range(10))
        A = array.array('i', L)
        A
```

```
Out[6]: array('i', [0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

■ 넘파이(Numpy) 사용

```
In [7]: import numpy as np
```

```
In [8]: # integer array:
        np.array([1, 4, 2, 5, 3])
```

```
Out[8]: array([1, 4, 2, 5, 3])
```

```
In [9]: np.array([3.14, 4, 2, 3])
```

```
Out[9]: array([ 3.14,  4. ,  2. ,  3. ])
```

데이터타입(DataType)

```
In [10]: np.array([1, 2, 3, 4], dtype='float32')
```

```
Out[10]: array([ 1.,  2.,  3.,  4.], dtype=float32)
```

```
In [11]: # nested lists result in multi-dimensional arrays
        np.array([range(i, i + 3) for i in [2, 4, 6]])
```

```
Out[11]: array([[2, 3, 4],
                [4, 5, 6],
                [6, 7, 8]])
```

파이썬 데이터 타입 이해하기 (6)

■ 넘파이 사용 배열 생성 예제들

```
In [12]: # Create a length-10 integer array filled with zeros
np.zeros(10, dtype=int)
```

```
Out[12]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

```
In [13]: # Create a 3x5 floating-point array filled with ones
np.ones((3, 5), dtype=float)
```

```
Out[13]: array([[ 1.,  1.,  1.,  1.,  1.],
 [ 1.,  1.,  1.,  1.,  1.],
 [ 1.,  1.,  1.,  1.,  1.]])
```

```
In [14]: # Create a 3x5 array filled with 3.14
np.full((3, 5), 3.14)
```

```
Out[14]: array([[ 3.14,  3.14,  3.14,  3.14,  3.14],
 [ 3.14,  3.14,  3.14,  3.14,  3.14],
 [ 3.14,  3.14,  3.14,  3.14,  3.14]])
```

```
In [15]: # Create an array filled with a linear sequence
# Starting at 0, ending at 20, stepping by 2
# (this is similar to the built-in range() function)
np.arange(0, 20, 2)
```

```
Out[15]: array([ 0,  2,  4,  6,  8, 10, 12, 14, 16, 18])
```

```
In [16]: # Create an array of five values evenly spaced between 0 and 1
np.linspace(0, 1, 5)
```

```
Out[16]: array([ 0. ,  0.25,  0.5 ,  0.75,  1.  ])
```

```
In [17]: # Create a 3x3 array of uniformly distributed
# random values between 0 and 1
np.random.random((3, 3))
```

```
Out[17]: array([[ 0.99844933,  0.52183819,  0.22421193],
 [ 0.08007488,  0.45429293,  0.20941444],
 [ 0.14360941,  0.96910973,  0.946117  ]])
```

```
In [18]: # Create a 3x3 array of normally distributed random values
# with mean 0 and standard deviation 1
np.random.normal(0, 1, (3, 3))
```

```
Out[18]: array([[ 1.51772646,  0.39614948, -0.10634696],
 [ 0.25671348,  0.00732722,  0.37783601],
 [ 0.68446945,  0.15926039, -0.70744073]])
```

```
In [19]: # Create a 3x3 array of random integers in the interval [0, 10]
np.random.randint(0, 10, (3, 3))
```

```
Out[19]: array([[2, 3, 4],
 [5, 7, 8],
 [0, 5, 0]])
```

```
In [20]: # Create a 3x3 identity matrix
np.eye(3)
```

```
Out[20]: array([[ 1.,  0.,  0.],
 [ 0.,  1.,  0.],
 [ 0.,  0.,  1.]])
```

```
In [21]: # Create an uninitialized array of three integers
# The values will be whatever happens to already exist at that m
np.empty(3)
```

```
Out[21]: array([ 1.,  1.,  1.] )
```

파이썬 데이터 타입 이해하기 (7)

■ 넘파이 표준 데이터 타입

Data type	Description
<code>bool_</code>	Boolean (True or False) stored as a byte
<code>int_</code>	Default integer type (same as C <code>long</code> ; normally either <code>int64</code> or <code>int32</code>)
<code>intc</code>	Identical to C <code>int</code> (normally <code>int32</code> or <code>int64</code>)
<code>intp</code>	Integer used for indexing (same as C <code>ssize_t</code> ; normally either <code>int32</code> or <code>int64</code>)
<code>int8</code>	Byte (-128 to 127)
<code>int16</code>	Integer (-32768 to 32767)
<code>int32</code>	Integer (-2147483648 to 2147483647)
<code>int64</code>	Integer (-9223372036854775808 to 9223372036854775807)
<code>uint8</code>	Unsigned integer (0 to 255)
<code>uint16</code>	Unsigned integer (0 to 65535)
<code>uint32</code>	Unsigned integer (0 to 4294967295)
<code>uint64</code>	Unsigned integer (0 to 18446744073709551615)
<code>float_</code>	Shorthand for <code>float64</code> .
<code>float16</code>	Half precision float: sign bit, 5 bits exponent, 10 bits mantissa
<code>float32</code>	Single precision float: sign bit, 8 bits exponent, 23 bits mantissa
<code>float64</code>	Double precision float: sign bit, 11 bits exponent, 52 bits mantissa
<code>complex_</code>	Shorthand for <code>complex128</code> .
<code>complex64</code>	Complex number, represented by two 32-bit floats
<code>complex128</code>	Complex number, represented by two 64-bit floats

```
np.zeros(10, dtype='int16')
```

```
np.zeros(10, dtype=np.int16)
```

정수형 타입(int16)의
0 값 10개인 배열

넘파이 (NumPy) 배열의 기초

넘파이 (NumPy) 배열의 기초 (1)

■ 넘파이 배열 속성 지정

- ndim: 차원의 개수
- shape: 각 차원의 크기
- size: 전체 배열의 크기
- dtype: 각 요소의 타입

```
import numpy as np
np.random.seed(0) # seed for reproducibility

x1 = np.random.randint(10, size=6) # One-dimensional array
x2 = np.random.randint(10, size=(3, 4)) # Two-dimensional array
x3 = np.random.randint(10, size=(3, 4, 5)) # Three-dimensional array
```

```
print("x3 ndim: ", x3.ndim)
print("x3 shape:", x3.shape)
print("x3 size: ", x3.size)
```

```
x3 ndim: 3
x3 shape: (3, 4, 5)
x3 size: 60
```

```
print("dtype:", x3.dtype)
```

```
dtype: int64
```

```
print("itemsizes:", x3.itemsize, "bytes")
print("nbytes:", x3.nbytes, "bytes")
```

```
itemsize: 8 bytes
nbytes: 480 bytes
```

넘파이 (NumPy) 배열의 기초 (2) - 인덱싱

- 배열 인덱싱 (단일 요소 접근)
 - 파이썬 리스트 접근과 비슷
 - '[' 과 ']' 사용
 - 다차원 배열 인덱스 구분은 comma(,) 사용

```
In [5]: x1
```

```
Out[5]: array([5, 0, 3, 3, 7, 9])
```

```
In [6]: x1[0]
```

```
Out[6]: 5
```

```
In [7]: x1[4]
```

```
Out[7]: 7
```

```
In [8]: x1[-1]
```

```
Out[8]: 9
```

```
In [9]: x1[-2]
```

```
Out[9]: 7
```

```
In [10]: x2
```

```
Out[10]: array([[3, 5, 2, 4],  
                [7, 6, 8, 8],  
                [1, 6, 7, 7]])
```

```
In [11]: x2[0, 0]
```

```
Out[11]: 3
```

```
In [12]: x2[2, 0]
```

```
Out[12]: 1
```

```
In [13]: x2[2, -1]
```

```
Out[13]: 7
```

```
In [14]: x2[0, 0] = 12
```

```
x2
```

```
Out[14]: array([[12, 5, 2, 4],  
                [ 7, 6, 8, 8],  
                [ 1, 6, 7, 7]])
```

```
In [15]: x1[0] = 3.14159 # this will  
x1
```

```
Out[15]: array([3, 0, 3, 3, 7, 9])
```

넘파이 (NumPy) 배열의 기초 (3) - 슬라이싱

■ 배열 슬라이싱 (하위 배열 접근)

- 콜론(:) 기호를 사용하여 슬라이스 (slice) 가능

```
x[start:stop:step]
```

■ 1차원 하위 배열

```
In [16]: x = np.arange(10)
x
```

```
Out[16]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [17]: x[:5] # first five elements
```

```
Out[17]: array([0, 1, 2, 3, 4])
```

```
In [18]: x[5:] # elements after index 5
```

```
Out[18]: array([5, 6, 7, 8, 9])
```

```
In [19]: x[4:7] # middle sub-array
```

```
Out[19]: array([4, 5, 6])
```

```
In [20]: x[::2] # every other element
```

```
Out[20]: array([0, 2, 4, 6, 8])
```

```
In [21]: x[1::2] # every other element, starting at index 1
```

```
Out[21]: array([1, 3, 5, 7, 9])
```

```
In [22]: x[::-1] # all elements, reversed
```

```
Out[22]: array([9, 8, 7, 6, 5, 4, 3, 2, 1, 0])
```

```
In [23]: x[5::-2] # reversed every other from index 5
```

```
Out[23]: array([5, 3, 1])
```

step 음수 사용 시 배열 거꾸로 !!

넘파이 (NumPy) 배열의 기초 (4) - 슬라이싱

- 다차원 하위 배열

- 코마로 구분된 다중 슬라이스를 사용

```
In [24]: x2
```

```
Out[24]: array([[12,  5,  2,  4],
                [ 7,  6,  8,  8],
                [ 1,  6,  7,  7]])
```

```
In [25]: x2[:2, :3] # two rows, three columns
```

```
Out[25]: array([[12,  5,  2],
                [ 7,  6,  8]])
```

```
In [26]: x2[:3, ::2] # all rows, every other column
```

```
Out[26]: array([[12,  2],
                [ 7,  8],
                [ 1,  7]])
```

```
In [27]: x2[::-1, ::-1]
```

```
Out[27]: array([[ 7,  7,  6,  1],
                [ 8,  8,  6,  7],
                [ 4,  2,  5, 12]])
```

```
In [28]: print(x2[:, 0]) # first column of x2
```

```
[12  7  1]
```

```
In [29]: print(x2[0, :]) # first row of x2
```

```
[12  5  2  4]
```

```
In [30]: print(x2[0]) # equivalent to x2[0, :]
```

```
[12  5  2  4]
```

넘파이 (NumPy) 배열의 기초 (5) - 슬라이싱

■ 사본이 아닌 뷰로서의 하위 배열

- 배열 슬라이스는 배열 데이터의 사본(copy)가 아니라 뷰(view)를 반환
- 파이썬 리스트에서의 슬라이스는 사본

In [31]: `print(x2)`

```
[[12  5  2  4]
 [ 7  6  8  8]
 [ 1  6  7  7]]
```

In [32]: `x2_sub = x2[:2, :2]`
`print(x2_sub)`

```
[[12  5]
 [ 7  6]]
```

In [33]: `x2_sub[0, 0] = 99`
`print(x2_sub)`

```
[[99  5]
 [ 7  6]]
```

In [34]: `print(x2)`

```
[[99  5  2  4]
 [ 7  6  8  8]
 [ 1  6  7  7]]
```

■ 배열의 사본 만들기

- `copy()` 메소드 사용

In [35]: `x2_sub_copy = x2[:2, :2].copy()`
`print(x2_sub_copy)`

```
[[99  5]
 [ 7  6]]
```

In [36]: `x2_sub_copy[0, 0] = 42`
`print(x2_sub_copy)`

```
[[42  5]
 [ 7  6]]
```

In [37]: `print(x2)`

```
[[99  5  2  4]
 [ 7  6  8  8]
 [ 1  6  7  7]]
```

넘파이 (NumPy) 배열의 기초 (6) - 재구조화

■ 배열의 재구조화

- 배열의 형상을 변경하는 것
- reshape() 메소드 사용

```
grid = np.arange(1, 10).reshape((3, 3))  
print(grid)
```

```
[[1 2 3]  
 [4 5 6]  
 [7 8 9]]
```

```
In [39]: x = np.array([1, 2, 3])  
  
# row vector via reshape  
x.reshape((1, 3))
```

```
Out[39]: array([[1, 2, 3]])
```

```
In [40]: # row vector via newaxis  
x[np.newaxis, :]
```

행 축 추가 !!

```
Out[40]: array([[1, 2, 3]])
```

```
In [41]: # column vector via reshape  
x.reshape((3, 1))
```

```
Out[41]: array([[1],  
               [2],  
               [3]])
```

```
In [42]: # column vector via newaxis  
x[:, np.newaxis]
```

```
Out[42]: array([[1],  
               [2],  
               [3]])
```

열 축 추가 !!

넘파이 (NumPy) 배열의 기초 (7) – 연결 및 분할

■ 배열 연결 (Concatenation)

- 여러 배열들을 하나로 결합
- `np.concatenate` (연결 붙임), `np.vstack` (수직으로 붙임), `np.hstack` (수평으로 붙임)

```
x = np.array([1, 2, 3])
y = np.array([3, 2, 1])
np.concatenate([x, y])

array([1, 2, 3, 3, 2, 1])
```

```
z = [99, 99, 99]
print(np.concatenate([x, y, z]))

[ 1  2  3  3  2  1 99 99 99]
```

```
grid = np.array([[1, 2, 3],
                  [4, 5, 6]])
```

```
# concatenate along the first axis
np.concatenate([grid, grid])
```

```
array([[1, 2, 3],
       [4, 5, 6],
       [1, 2, 3],
       [4, 5, 6]])
```

```
# concatenate along the second axis
np.concatenate([grid, grid], axis=1)
```

```
array([[1, 2, 3, 1, 2, 3],
       [4, 5, 6, 4, 5, 6]])
```

```
x = np.array([1, 2, 3])
grid = np.array([[9, 8, 7],
                  [6, 5, 4]])
```

```
# vertically stack the arrays
np.vstack([x, grid])
```

```
array([[1, 2, 3],
       [9, 8, 7],
       [6, 5, 4]])
```

```
# horizontally stack the arrays
y = np.array([[99],
               [99]])
np.hstack([grid, y])
```

```
array([[ 9,  8,  7, 99],
       [ 6,  5,  4, 99]])
```


넘파이 (NumPy) 배열의 기초 (8) – 연결 및 분할

■ 배열 분할 (Splitting)

- 하나의 배열을 여러 개의 배열로 분할
- `np.split` (분할), `np.hsplit` (수평 분할), `np.vsplit` (수직 분할)

```
x = [1, 2, 3, 99, 99, 3, 2, 1]
x1, x2, x3 = np.split(x, [3, 5])
print(x1, x2, x3)
```

```
[1 2 3] [99 99] [3 2 1]
```

```
grid = np.arange(16).reshape((4, 4))
grid
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15]])
```

```
upper, lower = np.vsplit(grid, [2])
print(upper)
print(lower)
```

```
[[0 1 2 3]
 [4 5 6 7]
 [ 8  9 10 11]
 [12 13 14 15]]
```

```
left, right = np.hsplit(grid, [2])
print(left)
print(right)
```

```
[[ 0  1]
 [ 4  5]
 [ 8  9]
 [12 13]]
[[ 2  3]
 [ 6  7]
 [10 11]
 [14 15]]
```

넘파이 (NumPy) 배열 연산: 유니버설 함수

넘파이 배열 연산: 유니버설 함수 (1)

- 넘파이 배열 연산: 유니버설 함수
 - 넘파이 배열 연산을 벡터화(Vectorized)하여 빠르게 처리
 - 유니버설 함수 (Universal functions, Ufuncs)를 통해 구현
- 파이썬의 반복 연산
 - 배열을 반복해서 각 요소를 조작하는 경우, 속도가 느림

```
import numpy as np
np.random.seed(0)

def compute_reciprocals(values):
    output = np.empty(len(values))
    for i in range(len(values)):
        output[i] = 1.0 / values[i]
    return output

values = np.random.randint(1, 10, size=5)
compute_reciprocals(values)
```

```
array([ 0.16666667,  1.          ,  0.25         ,  0.25         ,  0.125        ])
```

```
big_array = np.random.randint(1, 100, size=1000000)
%timeit compute_reciprocals(big_array)
```

1 loop, best of 3: 2.91 s per loop

넘파이 배열 연산: 유니버설 함수 (2)

▪ Ufuncs 소개

- 벡터화 연산: 여러 종류의 연산에 대해 정적 타입의 체계를 가진 컴파일러 루틴에 편리한 인터페이스 제공
- 배열에 요소에 적용하여 수행

```
print(compute_reciprocals(values))  
print(1.0 / values)
```

```
[ 0.16666667  1.          0.25          0.25          0.125         ]  
[ 0.16666667  1.          0.25          0.25          0.125         ]
```

```
%timeit (1.0 / big_array)
```

100 loops, best of 3: 4.6 ms per loop

```
np.arange(5) / np.arange(1, 6)
```

```
array([ 0.          ,  0.5          ,  0.66666667,  0.75          ,  0.8          ])
```

```
x = np.arange(9).reshape((3, 3))  
2 ** x
```

```
array([[ 1,  2,  4],  
       [ 8, 16, 32],  
       [64, 128, 256]])
```

넘파이 배열 연산: 유니버설 함수 (3) - 산술연산

- 유니버설 함수 (Ufuncs)

- 단일 입력 값에 동작하는 **단항 Ufuncs**와 두 개의 입력 값에 동작하는 **이항 Ufuncs**

- 배열 산술 연산

- Ufuncs는 파이썬의 기본 산술연산자를 사용하여 연산 (덧셈, 뺄셈, 곱셈, 나눗셈, 음수, 지수, 나머지 등)

```
x = np.arange(4)
print("x      =", x)
print("x + 5 =", x + 5)
print("x - 5 =", x - 5)
print("x * 2 =", x * 2)
print("x / 2 =", x / 2)
print("x // 2 =", x // 2) # floor division
```

```
x      = [0 1 2 3]
x + 5 = [5 6 7 8]
x - 5 = [-5 -4 -3 -2]
x * 2 = [0 2 4 6]
x / 2 = [ 0.  0.5  1.  1.5]
x // 2 = [0 0 1 1]
```

```
print("-x      = ", -x)
print("x ** 2 = ", x ** 2)
print("x % 2  = ", x % 2)
```

```
-x      = [ 0 -1 -2 -3]
x ** 2 = [0 1 4 9]
x % 2  = [0 1 0 1]
```

```
-(0.5*x + 1) ** 2
```

```
array([-1.  , -2.25, -4.  , -6.25])
```

넘파이 배열 연산: 유니버설 함수 (4) - 산술연산

■ 배열 산술 연산 (2)

- 산술 연산은 모두 사용상 편의를 위해 NumP에 내장된 특정 함수를 감싼 것 (Wrapping)

```
np.add(x, 2)
```

```
array([2, 3, 4, 5])
```

Operator Equivalent ufunc Description

+	<code>np.add</code>	Addition (e.g., <code>1 + 1 = 2</code>)
-	<code>np.subtract</code>	Subtraction (e.g., <code>3 - 2 = 1</code>)
-	<code>np.negative</code>	Unary negation (e.g., <code>-2</code>)
*	<code>np.multiply</code>	Multiplication (e.g., <code>2 * 3 = 6</code>)
/	<code>np.divide</code>	Division (e.g., <code>3 / 2 = 1.5</code>)
//	<code>np.floor_divide</code>	Floor division (e.g., <code>3 // 2 = 1</code>)
**	<code>np.power</code>	Exponentiation (e.g., <code>2 ** 3 = 8</code>)
%	<code>np.mod</code>	Modulus/remainder (e.g., <code>9 % 4 = 1</code>)

넘파이 배열 연산: 유니버설 함수 (5) - 절댓값

■ 절대값

- 산술 연산은 모두 사용상 편의를 위해 NumPy에 내장된 특정 함수를 감싼 것 (Wrapping)

```
x = np.array([-2, -1, 0, 1, 2])  
abs(x)  
  
array([2, 1, 0, 1, 2])
```

← 파이썬 내장 산술 연산자도 사용 가능!!

```
np.absolute(x)  
  
array([2, 1, 0, 1, 2])
```

```
np.abs(x)  
  
array([2, 1, 0, 1, 2])
```

```
x = np.array([3 - 4j, 4 - 3j, 2 + 0j, 0 + 1j])  
np.abs(x)  
  
array([ 5.,  5.,  2.,  1.])
```

← 복소수도 사용 가능

절댓값 ufuncs

넘파이 배열 연산: 유니버설 함수 (6) - 삼각함수

■ 삼각 함수

```
theta = np.linspace(0, np.pi, 3)
```

```
print("theta      = ", theta)
print("sin(theta) = ", np.sin(theta))
print("cos(theta) = ", np.cos(theta))
print("tan(theta) = ", np.tan(theta))
```

```
theta      = [ 0.          1.57079633  3.14159265]
sin(theta) = [ 0.00000000e+00  1.00000000e+00  1.22464680e-16]
cos(theta) = [ 1.00000000e+00  6.12323400e-17 -1.00000000e+00]
tan(theta) = [ 0.00000000e+00  1.63312394e+16 -1.22464680e-16]
```

```
x = [-1, 0, 1]
print("x          = ", x)
print("arcsin(x) = ", np.arcsin(x))
print("arccos(x) = ", np.arccos(x))
print("arctan(x) = ", np.arctan(x))
```

```
x          = [-1, 0, 1]
arcsin(x) = [-1.57079633  0.          1.57079633]
arccos(x) = [ 3.14159265  1.57079633  0.          ]
arctan(x) = [-0.78539816  0.          0.78539816]
```


넘파이 배열 연산: 유니버설 함수 (7) – 지수/로그

■ 지수와 로그

- `np.exp` : 지수
- `np.log`: 자연로그, `np.log2`, `np.log10`

```
x = [1, 2, 3]
print("x      =", x)
print("e^x    =", np.exp(x))
print("2^x    =", np.exp2(x))
print("3^x    =", np.power(3, x))
```

```
x      = [1, 2, 3]
e^x    = [ 2.71828183  7.3890561 20.08553692]
2^x    = [ 2.  4.  8.]
3^x    = [ 3  9 27]
```

```
x = [1, 2, 4, 10]
print("x      =", x)
print("ln(x)   =", np.log(x))
print("log2(x)  =", np.log2(x))
print("log10(x) =", np.log10(x))
```

```
x      = [1, 2, 4, 10]
ln(x)   = [ 0.          0.69314718  1.38629436  2.30258509]
log2(x) = [ 0.          1.          2.          3.32192809]
log10(x) = [ 0.          0.30103    0.60205999  1.          ]
```

```
x = [0, 0.001, 0.01, 0.1]
print("exp(x) - 1 =", np.expm1(x))
print("log(1 + x) =", np.log1p(x))
```

```
exp(x) - 1 = [ 0.          0.0010005  0.01005017  0.10517092]
log(1 + x) = [ 0.          0.0009995  0.00995033  0.09531018]
```

← **x가 매우 작을때 사용**

넘파이 배열 연산: 유니버설 함수 (8) – 특화 ufuncs

■ 특화된 유니버설 함수

- 쌍곡선 삼각 함수, 비트 연산, 비교 연산자, 라디안을 각도로 변환, 반올림/나머지 등
- `scipy.special` : 잘 알려지지 않은 수학적 함수 사용

```
from scipy import special
```

```
# Gamma functions (generalized factorials) and related functions
```

```
x = [1, 5, 10]
```

```
print("gamma(x)      =", special.gamma(x))
```

```
print("ln|gamma(x)| =", special.gammaln(x))
```

```
print("beta(x, 2)   =", special.beta(x, 2))
```

```
gamma(x)      = [ 1.00000000e+00  2.40000000e+01  3.62880000e+05]
```

```
ln|gamma(x)| = [ 0.          3.17805383 12.80182748]
```

```
beta(x, 2)   = [ 0.5          0.03333333 0.00909091]
```

```
# Error function (integral of Gaussian)
```

```
# its complement, and its inverse
```

```
x = np.array([0, 0.3, 0.7, 1.0])
```

```
print("erf(x)  =", special.erf(x))
```

```
print("erfc(x) =", special.erfc(x))
```

```
print("erfinv(x) =", special.erfinv(x))
```

```
erf(x)  = [ 0.          0.32862676 0.67780119 0.84270079]
```

```
erfc(x) = [ 1.          0.67137324 0.32219881 0.15729921]
```

```
erfinv(x) = [ 0.          0.27246271 0.73286908      inf]
```

넘파이 배열 연산: 유니버설 함수 (9) – 고급 Ufunc 기능

■ 고급 Ufunc 기능 - 출력 지정

- 대규모 연산 시, 연산 결과를 저장할 배열을 지정
- 모든 Ufuncs에서 함수의 out 인수를 사용해 출력 지정

```
x = np.arange(5)
y = np.empty(5)
np.multiply(x, 10, out=y)
print(y)
```

```
[ 0. 10. 20. 30. 40.]
```

```
y = np.zeros(10)
np.power(2, x, out=y[::2])
print(y)
```

```
[ 1.  0.  2.  0.  4.  0.  8.  0. 16.  0.]
```

넘파이 배열 연산: 유니버설 함수 (10) – 고급 Ufunc 기능

▪ 고급 Ufunc 기능 - 집계

- 배열을 특정 연산으로 축소 한다면 → reduce 메소드 사용
- reduce 메소드는 결과가 하나만 남을 때까지 해당 연산을 배열 요소에 반복해서 적용

```
x = np.arange(1, 6)  
np.add.reduce(x)
```

15

```
np.multiply.reduce(x)
```

120

```
np.add.accumulate(x)
```

array([1, 3, 6, 10, 15])

```
np.multiply.accumulate(x)
```

array([1, 2, 6, 24, 120])

넘파이 배열 연산: 유니버설 함수 (11) – 고급 Ufunc 기능

- 고급 Ufunc 기능 – 외적 (Outer products)

- outer 메소드: 서로 다른 두 입력 값의 모든 쌍에 대한 출력값을 계산

```
x = np.arange(1, 6)
np.multiply.outer(x, x)
```

```
array([[ 1,  2,  3,  4,  5],
       [ 2,  4,  6,  8, 10],
       [ 3,  6,  9, 12, 15],
       [ 4,  8, 12, 16, 20],
       [ 5, 10, 15, 20, 25]])
```

집계: 최솟값, 최댓값 등

집계: 최솟값, 최댓값 등 (1)

- 배열의 값의 합 구하기
 - np.sum 사용

```
import numpy as np
```

```
L = np.random.random(100)  
sum(L)
```

55.61209116604941

파이썬 내장함수(sum) 사용

```
np.sum(L)
```

55.612091166049424

넘파이 np.sum 사용

```
big_array = np.random.rand(1000000)  
%timeit sum(big_array)  
%timeit np.sum(big_array)
```

10 loops, best of 3: 104 ms per loop

1000 loops, best of 3: 442 μ s per loop

파이썬 내장함수(sum) vs 넘파이 함수(np.sum) 속도 비교

집계: 최솟값, 최댓값 등 (2)

- 최솟값 및 최댓값

- np.min 및 np.max 사용

```
min(big_array), max(big_array)
```

(1.1717128136634614e-06, 0.9999976784968716)

파이썬 내장함수(min,max) 사용

```
np.min(big_array), np.max(big_array)
```

(1.1717128136634614e-06, 0.9999976784968716)

넘파이 np.min/np.max 사용

```
%timeit min(big_array)  
%timeit np.min(big_array)
```

10 loops, best of 3: 82.3 ms per loop

1000 loops, best of 3: 497 µs per loop

파이썬 내장함수(min) vs 넘파이 함수(np.min) 속도 비교

집계: 최솟값, 최댓값 등 (3)

■ 다차원 집계

- 기본적으로 전체 배열의 집계 값을 반환

```
M = np.random.random((3, 4))  
print(M)
```

```
[[ 0.8967576  0.03783739  0.75952519  0.06682827]  
 [ 0.8354065  0.99196818  0.19544769  0.43447084]  
 [ 0.66859307  0.15038721  0.37911423  0.6687194 ]]
```

```
M.sum()
```

```
6.0850555667307118
```

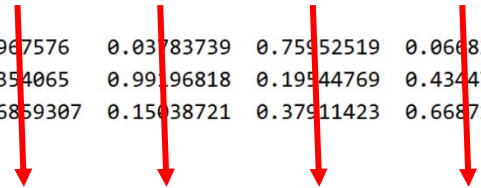
- 어느 축(axis)을 따라 집계할 것인지를 지정 가능

```
M.min(axis=0)
```

```
array([ 0.66859307,  0.03783739,  0.19544769,  0.06682827])
```

```
M.max(axis=1)
```

```
array([ 0.8967576 ,  0.99196818,  0.6687194 ])
```



```
[[ 0.8967576  0.03783739  0.75952519  0.06682827]  
 [ 0.8354065  0.99196818  0.19544769  0.43447084]  
 [ 0.66859307  0.15038721  0.37911423  0.6687194 ]]
```



```
[[ 0.8967576  0.03783739  0.75952519  0.06682827]  
 [ 0.8354065  0.99196818  0.19544769  0.43447084]  
 [ 0.66859307  0.15038721  0.37911423  0.6687194 ]]
```

집계: 최솟값, 최댓값 등 (4)

- 기타 집계 함수

- 누락된 값을 무시한 채 값을 계산하는 NaN 안전 모드 존재

Function Name	NaN-safe Version	Description
<code>np.sum</code>	<code>np.nansum</code>	Compute sum of elements
<code>np.prod</code>	<code>np.nanprod</code>	Compute product of elements
<code>np.mean</code>	<code>np.nanmean</code>	Compute mean of elements
<code>np.std</code>	<code>np.nanstd</code>	Compute standard deviation
<code>np.var</code>	<code>np.nanvar</code>	Compute variance
<code>np.min</code>	<code>np.nanmin</code>	Find minimum value
<code>np.max</code>	<code>np.nanmax</code>	Find maximum value
<code>np.argmin</code>	<code>np.nanargmin</code>	Find index of minimum value
<code>np.argmax</code>	<code>np.nanargmax</code>	Find index of maximum value
<code>np.median</code>	<code>np.nanmedian</code>	Compute median of elements
<code>np.percentile</code>	<code>np.nanpercentile</code>	Compute rank-based statistics of elements
<code>np.any</code>	N/A	Evaluate whether any elements are true
<code>np.all</code>	N/A	Evaluate whether all elements are true

(예제) 미국 대통령의 평균 신장은?

(예제) 미국 대통령의 평균 신장은 ? (1)

- (예제)

- president_heights.csv 파일 사용

```
!head -4 data/president_heights.csv
```

```
order,name,height(cm)
1,George Washington,189
2,John Adams,170
3,Thomas Jefferson,189
```

- pandas 패키지 사용 (추후 배움)

```
import pandas as pd
data = pd.read_csv('data/president_heights.csv')
heights = np.array(data['height(cm)'])
print(heights)
```

```
[189 170 189 163 183 171 185 168 173 183 173 173 175 178 183 193 178 173
 174 183 183 168 170 178 182 180 183 178 182 188 175 179 183 193 182 183
 177 185 188 188 182 185]
```

- 다양한 통계 추출

```
print("Mean height:      ", heights.mean())
print("Standard deviation:", heights.std())
print("Minimum height:   ", heights.min())
print("Maximum height:   ", heights.max())
```

```
Mean height:      179.738095238
Standard deviation: 6.93184344275
Minimum height:   163
Maximum height:   193
```

(예제) 미국 대통령의 평균 신장은 ? (2)

■ (예제)

- 사분위수 계산 가능

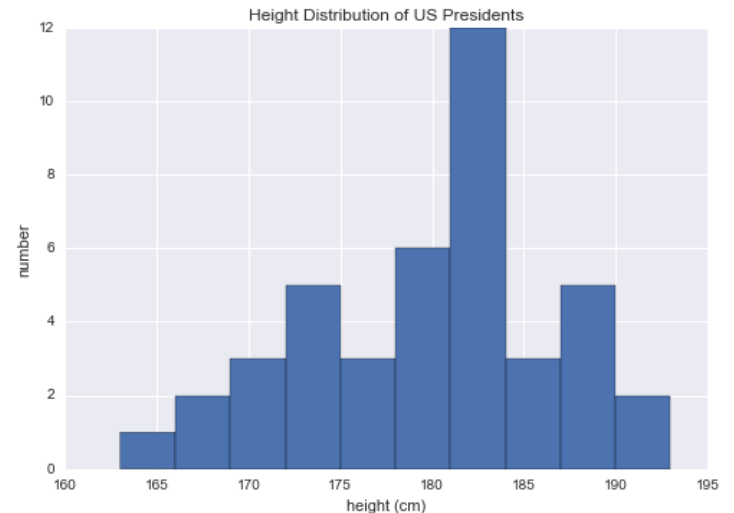
```
print("25th percentile: ", np.percentile(heights, 25))
print("Median:          ", np.median(heights))
print("75th percentile: ", np.percentile(heights, 75))
```

```
25th percentile:    174.25
Median:             182.0
75th percentile:    183.0
```

- matplotlib 사용하여 시각화

```
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn; seaborn.set() # set plot style
```

```
plt.hist(heights)
plt.title('Height Distribution of US Presidents')
plt.xlabel('height (cm)')
plt.ylabel('number');
```



Q & A
